TU Dortmund

Internship report

Text classification using BERT- Bidirectional Encoder Representations from Transformers

Lecturers:
Prof. Dr. Carsten Jentsch
M. Sc. Kai-Robin Lange

Author: Ankita Sarkar (230463)

March 30, 2023

Contents

1 Introduction **3**

2 Problem statement **3**

3 Machine Learning Models **5**

3.1 Previous related work 6

3.1.1 Word Embedding 6

3.1.2 Seq2seq model/Encoder-Decoder model 8

3.1.3 Attention 8

3.1.4 Transformers 10

3.1.5 Universal Language Model Fine-tuning: ULMFiT 12

3.1.6 OpenAI Transformer: GPT 13

3.2 BERT Model Architecture 13

3.3 Natural Language Toolkit - NLTK 16

3.4 Transformer-interpret 16

3.5 FastText 17

3.6 Confusion matrix 17

4 Result analysis **19**

4.1 Preprocessing of the data 19

4.2 Analysis of contributions of word attributions in lightly processed text and fully preprocessed text 21

4.3 Model performances 25

4.3.1 Parameters for fine-tuning BERT 25

4.3.2 Using lightly processed text 26

4.3.3 Using fully preprocessed text 27

5 Summary **29**

Bibliography **31**

Appendix **33**

1 Introduction

Generali is a leading insurance group in the German market with over 9 million customers and a premium income of 14.9 billion euros. The company operates in the life, health, and property/casualty segments under the brands Generali, CosmosDirekt, and Dialog and is part of the wider international Generali Group. To access Generali Deutschland Versicherungen's products, customers can exclusively turn to the Deutsche Vermögensberatung group of companies. Generali strives to be a lifelong partner for its customers by providing personalized and innovative solutions and services. Its strong sales network spans exclusive and direct sales as well as the broker channel. Generali Deutschland receives approx. 1 million claims per year.

This project is carried out by the Data Analytics team of the insurance company Generali Deutschland AG. One of the main ongoing topic is the Claims Automation project. Currently, the customer or an insurance agent notifies the company of a claim which is then manually entered into the system by filling in the form fields required to process the claim. Next, the claim is further verified and the customer is compensated. The Claims Automation project aims to automate the entire process by implementing a robotic automation process (RPA). Currently, the incoming claims are being classified using the classical statistical language models. The state-of-the-art neural network models owing to their ability to well generalization achieve better results than the traditional language models. BERT (Bidirectional Encoder Representations from Transformers) is a publicly available language model developed by Google in 2018 that revolutionized natural language processing (NLP). The goal of this project is to add the BERT model to the classification of the claims.

In Section 2 of the project report, the dataset and objectives of the project are presented. Section 3 discusses various machine learning models that are relevant to the project. The exploratory data analysis, text pre-processing steps and the prediction outcomes of the desired field are outlined in Section 4. Finally, the project summary is provided along with the scope for further development.

2 Problem statement

Generali mainly receives claims through email or letters. Initially, the claims information have been manually read and entered into the system by employees handling the claims

area. To make the process efficient and save resources, the digital data analytics team has undertaken a project to automate the whole process. As part of the project, the claims notification documents are first scanned and stored as pdf documents in a digital archive. Then a text-extraction service extracts the long raw text from the OCR version of the pdf by using the document-id of each claim. The raw text along with several other business fields such as doc-id, obgr(claim-id), sd-anlage-dat (claims notifying date), schadentag (damage date), sparte (damage type), schaden-objekt (damage sub-type), schaden-typ-kennung (type of schaden-objekt), notifier and schaden-urs-art (cause of damage) make up a data frame for further processing. The raw text is being prepossessed for further model training. Preprocessing steps include converting the text to lowercase, removing special characters, substituting multiple spaces with a single space, removing single characters, removing digits, lemmatization, expanding german contractions and replacing german special characters. The preprocessed text and the above mentioned business fields which serve as labels in the supervised model training are stored in the HIVE database. Figure 1 gives an overview of this process.

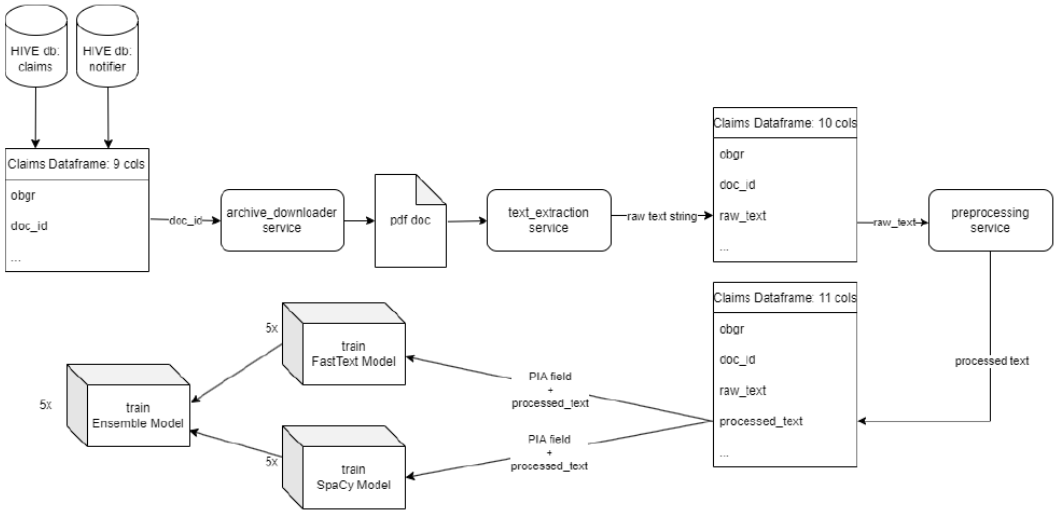


Figure 1: Claims Automation process in development.

Different text classifiers are trained on the labeled data frame so that they can predict the fields properly in production where the labels are not available as shown in figure 2. The classification models being used are fatstext, spacy, and an ensemble model consisting of Random Forest, Logistic Regression, XGBoost and Bagging Classifiers. This study aims to implement the BERT model to classify the fields. Depending on the performances of the BERT model, further plans involve to include BERT in the ensemble model for better classification results. The use case of this project only focuses

on a particular field sd-typ-kennung for BERT prediction. This field has three categories namely leitungswasser (LW) claims, storm (ST) and other claims (including glass, fire, bike theft, etc).

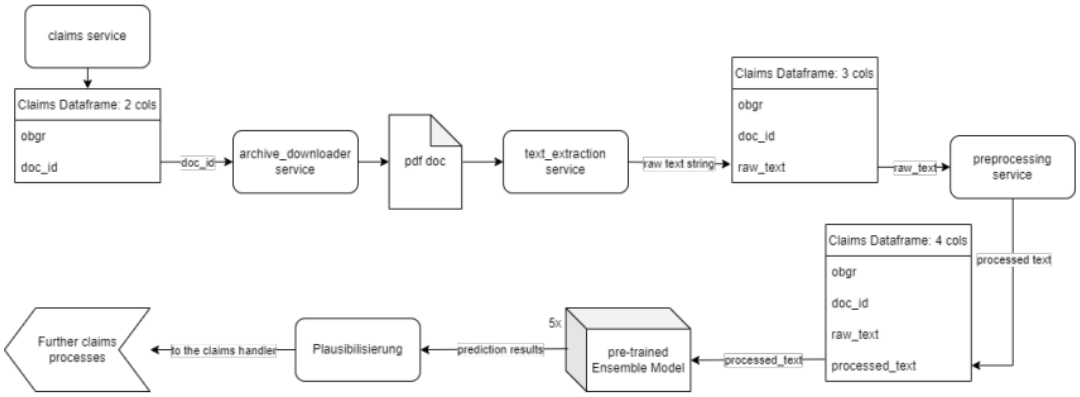


Figure 2: Claims Automation process in production.

A total of $1,57,881$ claims records are received by the company so far. As proof of concept for this project, only around $25,000$ (exactly $26,468$) records are being used. Out of which $14,742$ records fall into the other claims, $5,883$ are leitungswasser (LW) claims and $5,843$ are storm (ST) claims. BERT's pre-trained model is publicly available making it easy for researchers and developers to use it in their projects. In this project, the BERT model is trained on a labeled dataset for claims classification of the field sd-typ-kennung with its three different classes (LW, ST, and other). To compare the newly implemented BERT model's performance, the already deployed fasttext model's predictions on the same field are used. Additionally, to gain insight into the BERT model's workings the exploration of words distribution in text are checked and the contributions of the word attributions of tokens generated towards model prediction are explored.

3 Machine Learning Models

In this section, various machine learning methods that serve as building blocks in the BERT model architecture and other analysis methods used to work with the use-case in the project are discussed. Python programming language version 3.8 is utilized and the databases used are HIVE and a digital archive loader. The packages employed include

nltk, pandas, numpy, sklearn, seaborn, matplotlib, torch, transformers and transformers-interpret.

3.1 Previous related work

3.1.1 Word Embedding

Language modeling is a common NLP task that involves predicting the probability distribution of the next character in a sequence based on the input. To achieve this, language models use word embedding algorithms to represent words as vectors. The words with similar meanings are given similar representations. These algorithms are trained on a predefined fixed-size vocabulary of a text corpus using real-valued vectors that are dense with tens or hundreds of dimensions as opposed to the sparse vectors used in one-hot encoding which have thousands or millions of dimensions. The goal of word embedding is to create low-dimensional word vectors that convert raw text into numeric form which can then be fed into a deep neural network. Efficient algorithms such as word2vec and GloVe are used to learn distributed word representations (Brownlee [2017]).

Word2vec Word2vec has been developed by Google in 2013 to generate word embeddings. As mentioned earlier the words are represented as vectors in machine learning models. Word2vec takes text corpus as input and produces word vectors as output building a vocabulary from the training corpus. These word vectors are used as features in other NLP tasks. The learning involves calculating the distances between two word vectors (for eg, the words can be 'France' and 'Spain'). Two algorithms are used in word2vec: continuous bag-of-words (CBOW) and continuous skip-gram. Both algorithms learn a word by focusing on its context. The context can be set by fixing a window on the neighboring words. Simple vector operations are able to find similarities in the word space. For e.g., $\mathbf{vector}(\mathbf{'Paris'}) - \mathbf{vector}(\mathbf{'France'}) + \mathbf{vector}(\mathbf{'Italy'})$ results in $\mathbf{vector}(\mathbf{'Rome'})$. Larger embeddings in the case of phrases can also be constructed using word2vec by using large corpora of text (billions of words) during training (code archive(code archive [2013]).

GloVe GloVe is a type of unsupervised learning algorithm that creates vector representations of words. It is based on the word2vec method which was developed as an

open-source project at Stanford. However, the way that GloVe and word2vec process the training corpus to create these vector representations differs resulting in slightly different properties between the two models. In general, GloVe produces a single scalar value to capture the nearest neighbors for similar words. For instance, the words "man" and "woman" can be considered similar because they both refer to human beings. However, they are not very similar because they are associated with different genders. Therefore, more than a single number is needed to accurately capture the relationship between the two words. To address this issue, GloVe constructs an explicit global word-word co-occurrence matrix by analyzing the entire training text corpus. This matrix shows how often each word appears with another word in the corpus. By analyzing this matrix, GloVe is able to generate vector representations of words that take into account their relationships with other words in the corpus. (Pennington et al. [2014]).

ELMo Embeddings from Language Models (ELMo) is a deep contextualized word representation developed by AllenNLP. ELMo computes the word vectors on top of a two-layer bidirectional language model (biLM). The LM is called bidirectional as it processes the input sequence from both left-to-right and right-to-left directions. The biLM uses two layers of Long-Short Term Memory (LSTM) stacked together. Each layer involves the forward pass and the backward pass. The input word vectors to the biLM are computed from characters rather than words. The forward pass contains a word and the context before it whereas the backward pass contains information about the word and the context after it. The intermediate word vectors are generated from the first layer and fed to the second layer which generates another set of intermediate word vectors. ELMo is thus a weighted summation of the raw and two intermediate word vectors. For example to capture the meaning of 'bank' - a financial institution or river bank, ELMo looks at an entire sentence and gives contextualized word embeddings instead of a fixed embedding(Peters et al. [2018]).

Wordpiece embeddings Wordpiece embedding works by breaking down words into smaller sub-word units called "wordpieces". These wordpieces are created by applying an algorithm that takes into account the frequency of different sequences of letters or characters in the language. The most common sequences are then chosen as wordpieces, which can be combined to form any word in the language. Once the wordpieces are created, they can be used to represent words in a text document as vectors of numbers. One advantage of wordpiece embeddings is that they can handle out-of-vocabulary

words, which are words that are not in the vocabulary of the model because they are rare or new. This is because rare or new words can be broken down into their constituent wordpieces, which the model will recognize and use to create a representation for the word. This makes wordpiece embeddings particularly useful for tasks such as machine translation, sentiment analysis, and text classification, where there is a large variation in vocabulary and new words can be introduced over time (Wu et al. [2016]).

3.1.2 Seq2seq model/Encoder-Decoder model

Sequence to sequence models are a type of recurrent neural networks (RNNs) that are useful in various natural language processing (NLP) tasks, including machine translation and question answering. The Seq2seq model is made up of an encoder and a decoder, each of which is a RNN. The encoder reads each item in an input sequence and creates a context vector that holds information from the entire input sequence. The context vector is then passed to the decoder, which produces the output sequence item by item. The size of the context vector can be determined by the number of hidden units in the encoder. The encoder is formed by concatenating a forward and a backward hidden state which includes both preceding and following words. Although the output of the encoder is discarded, the state is kept to serve as the context for the decoder in the next step. However, the limited size of the context vector fails to capture all the past information as the input sequence gets longer. To overcome this issue, attention mechanisms (described below) are added to the Seq2seq model. (Alammar [2018]).

3.1.3 Attention

In the seq2seq model, the initial hidden layer of the decoder is connected to only the final hidden state of the encoder which may cause it to forget some past information as the input sequence gets longer. To address this issue, the decoder can access all the hidden layers of the encoder. The attention mechanism allows the decoder to selectively focus on the relevant information in a long sequence, similar to how humans read long sentences. Attention is a weight vector that assigns higher scores to certain hidden states of the encoder. After the scores are normalized, the hidden states with higher scores are emphasized while those with lower scores are suppressed in the output. The decoder incorporates an additional single-layer feed-forward neural network to compute

the relevance of the encoder state at a given time step. The following provides the mathematical definition of the attention mechanism (Bahdanau et al. [2014]).

$$\mathbf{x} = [x_1, x_2, \cdots, x_n] \qquad \text{,input sequence of length n}$$
$$\mathbf{y} = [y_1, y_2, \cdots, y_n] \qquad \text{,output sequence of length m}$$

The encoder state which is concatenation of a forward pass $\vec{h_i}$ and a backward pass $\overleftarrow{h_i}$ as below

$$h_i = [\vec{h_i}^\top; \overleftarrow{h_i}]^\top \qquad ,i = 1, \cdots, n$$

The hidden state in the decoder is as below

$$s_t = f(s_{t-1}, y_{t-1}, c_t) \text{ at time step, } t = 1, \cdots, m;\ c_t, \text{ context vector}$$

The context vector for the output $y_i, i = 1, \cdots, n$ is calculated as below

$$c_t = \sum_{i=1}^{n} \alpha_{t,i} h_i;\ t = 1, \cdots, m;\ i = 1, \cdots, n$$

$$\begin{aligned} \alpha_{t,i} &= align(y_i, x_i) \\ &= \frac{\exp(score(s_{t-1}, h_i))}{\sum_{i'=1}^{n} \exp(score(s_{t-1}, h_{i'}))} \end{aligned} \tag{1}$$

The score $\alpha_{t,i}$ is the alignment between the input at position i and the output at position t (y_i, x_i) depending on how they match. The score function in equation 1 is given by

$$score(s_t, h_i) = v_a^(W_a[s_t; h_i])$$

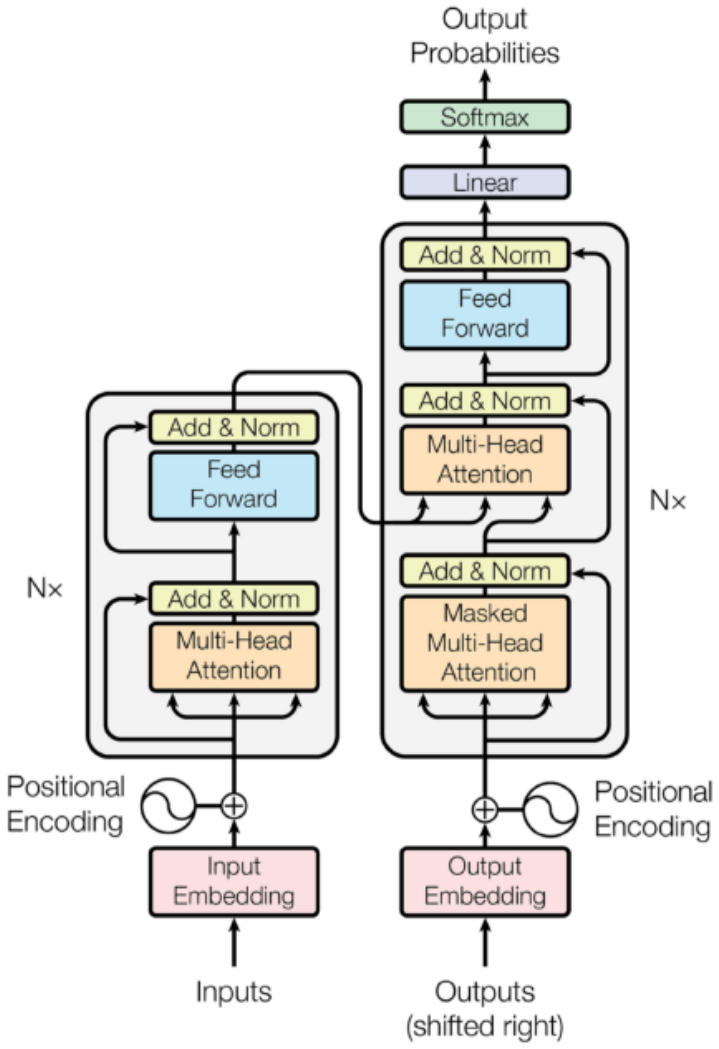
v_a and w_a are weight matrices for the alignment model.

3.1.4 Transformers

The traditional RNN/LSTM based approaches have limitations when it comes to processing long sequences as they are unable to parallelize the network effectively. Transformers is a novel neural network which has been proven to overcome this limitations because they can handle long-term dependencies more effectively. The transformer architecture consists of a stack of identical encoders and decoders. In contrast, BERT's architecture does not include decoders and instead relies solely on encoders. To understand the principles behind BERT, it is important to have a basic understanding of encoders and the attention mechanism. Each encoder is composed of two sub-layers: a self-attention layer that enables the encoder to consider other words in the input sentence as it encodes a specific word and a feed-forward neural network. The bottom-most encoder is responsible for the embedding process while all encoders receive a list of 512-sized vectors. The length of the list is determined by the maximum sentence length in the training dataset. Each word in the input sequence goes through the two layers of the encoder. Specifically, each word flows through its own path in the encoder while there are dependencies between these paths in the self-attention layer, the feed-forward layer doesn't have such dependencies. Therefore, the paths can be executed in parallel through the feed-forward layer.

In the Transformer architecture, the self-attention mechanism allows the model to incorporate the context of other relevant words into the word currently being processed. As the model processes each word in the input sequence, self-attention enables it to examine other positions in the sequence to gather the information that can help create a better encoding for the current word. This helps the model to build a better understanding of the input sequence as a whole. The output from the self-attention layer is then fed through a feed-forward neural network before being passed on to the next encoder. The encoder is responsible for transforming the original input sequence into a latent representation using hidden state vectors. In the case of sentence translation, the input is a sentence with each word's embedding and relative position represented by a positional encoding. The encoder utilizes a multi-head attention layer allowing the model to understand which words are related and their contextual meanings. This layer outputs an updated word embedding of the same size that considers its context. The updated embedding is normalized and then passed through a feed-forward neural network with two linear layers and an activation function.

The attention mechanism is a process that takes two sentences and transforms them into a matrix where one sentence's words form the rows and the other sentence's words form the columns. Transformers utilize the self-attention concept where the rows and columns are from the same sentence. Self-attention allows the model to learn the relationships between words in and across sentences. For instance, in the sentence "The animal didn't cross the street. It was too tired," self-attention enables the model to associate "it" with "animal." However, the distance that models can travel back to connect words is limited by the model's design. In the case of BERT, it can link words that are up to 512 sequences apart but this capability requires significant computing resources.



Source: Vaswani et al. [2017]
Figure 3: The transformer model architecture.

During the training phase, the Transformer model learns three new embeddings for each word in the input sentence: a query embedding (Q_i), a key embedding (K_i) and a value embedding (V_i). Each word embedding (X_i) queries every other word embedding (X_j) to determine its relative importance. This is done by taking the dot product of Q_i and K_i dividing it by the square root of the key matrix dimension d_k and passing the result through a softmax function to obtain a weight ($S_{i,j}$) for each word. The new embedding $V_{0,i}$ is then calculated by multiplying each weight with its corresponding value embedding (V_j) and summing them up.

$$S_{i,j} = softmax(\frac{Q_i {K_i}^\top}{\sqrt{d_k}})$$
$$V_{0,i} = S_{i,j} V_j$$

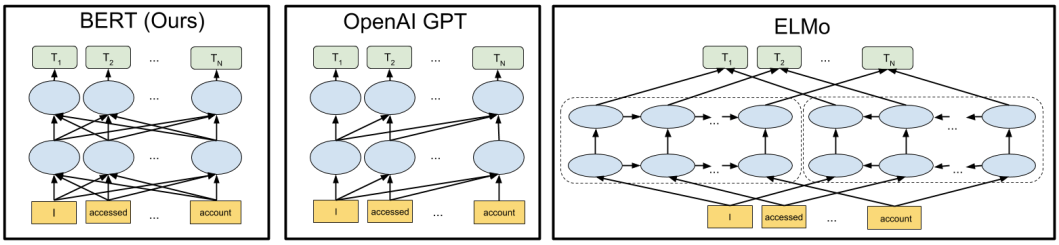
Since words can have different meanings in different contexts, multiple sets of queries, keys and values are used to create h different final embeddings ($V_{0,i}$) for each word (X_i). These embeddings are concatenated and multiplied by a learned matrix (Z) to reduce the concatenated embedding to the original size resulting in a final transformed embedding for each word. This process is called multi-head attention and it allows the model to consider multiple contexts for each word and address the issue of static word embeddings (Vaswani et al. [2017]).

3.1.5 Universal Language Model Fine-tuning: ULMFiT

ULMFiT is a transfer learning method for natural language processing (NLP) tasks. The ULMFiT approach involves pretraining a language model on a large corpus of text and then fine-tuning it on a smaller dataset for a specific NLP task such as sentiment analysis, text classification or named entity recognition. This is achieved by adding a task-specific output layer to the pre-trained language model and training the entire model end-to-end on the task-specific data. During fine-tuning, the pre-trained language model is used as a starting point and is further trained on the task-specific data with a much smaller learning rate to prevent catastrophic forgetting. The task-specific output layer is added on top of the pre-trained model and it is trained using the labeled data of the specific task (Howard and Ruder [2018]).

3.1.6 OpenAI Transformer: GPT

ULMFiT successfully bridged the gap between computer vision and NLP using LSTM-based transfer learning but the researchers felt that there was room for improvement. The emergence of transformers in 2017 led to better results than LSTM architectures. The LSTM is replaced with decoders resulting in further top-performing models across various tasks. OpenAI's GPT model introduced a fine-tunable pre-trained model based on the decoder portion of transformers but the transition from LSTMs to decoders caused a loss of bidirectionality in the language model. ELMo's bidirectional language model was able to train separately on the two tasks allowing it to resolve this issue but BERT took a different approach by training a masked language model as discussed below. This approach allowed BERT to effectively learn contextual word embeddings bidirectionally as depiceted in figure 4 (Radford et al. [2018]).



Source: Devlin et al. [2018]
Figure 4: Differences in pre-training BERT, GPT and ELMo model architectures.

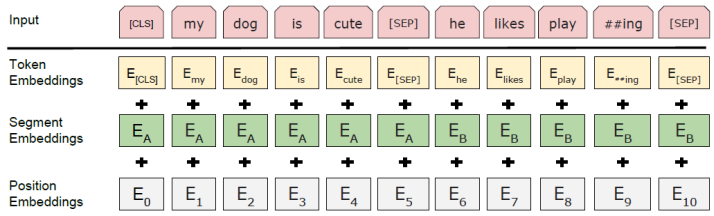
3.2 BERT Model Architecture

BERT, which stands for Bidirectional Encoder Representations from Transformers, is a language representation model that consists of two stages: pre-training and fine-tuning. It is the first model of its kind to learn deep bidirectional representations from the unlabelled text on both the left and right context in all layers. This pre-trained model can then be fine-tuned with minimal changes to the architecture on a variety of downstream tasks such as language inference and question-answering, by adding only an additional output layer. BERT's unique feature lies in the minimal differences between the pre-trained and fine-tuned models.

BERT is built upon a number of previous works such as Semi-supervised Sequence Learning, ELMo, ULMFiT, the OpenAI transformer and the Transformer to name a few.

BERT is a trained transformer encoder stack. The original paper introduces two models of two different sizes: $BERT_{BASE}$ with 12 transformer blocks (L), 768 hidden units in feed-forward networks (H), and 12 self-attention heads (A) whereas $BERT_{LARGE}$ has $L = 24, H = 1024$ and $A = 16$ (Devlin et al. [2018]).

Model inputs The BERT model accepts one or two sentences as input tokens which can be utilized for various downstream tasks. The training of the model involves using arbitrary sentences rather than the meaningful ones. The authors employed WordPiece embeddings with a vocabulary of 30,000 tokens. The first token of each sequence is [CLS] which denotes the classification tasks. Since two sentences are combined in a sequence, another token [SEP] is utilized to differentiate between them. Additionally, a learned embedding is added to each token to indicate whether it comes from sentence A or B. The input representation is formed by adding token, segment, and position embeddings as illustrated in figure 5.



Source: Devlin et al. [2018]
Figure 5: BERT input representation.

To pre-train the BERT model, the existing literature on language model pretraining such as $800M$ words from BooksCorpus (Zhu et al. [2015]) and $2,500M$ words from English Wikipedia (ignoring the lists, tables and headers) are used (Devlin et al. [2018]).

Pre-training of BERT BERT has been pre-trained using two unsupervised tasks: Maksed LM and Next Sentence Prediction (NSP) tasks.

Task1: Masked Language Model (LM) Traditional language models can only be trained in a unidirectional manner, either from left-to-right or right-to-left. This is because predicting a target word in a multi-layered context indirectly while being trained bidirectionally can be difficult. To address this issue, BERT pre-training applies masking to a random selection of 15% of input tokens in each sequence and attempts to predict these masked tokens by feeding the corresponding final hidden vectors to an output

softmax layer which is referred to as Masked LM (MLM). However, during fine-tuning, the [MASK] tokens are not present resulting in mismatches. To make the fine-tuning of the MLM more robust, various masking strategies are employed such as replacing masked words with the [MASK] token 80% of the time, replacing the selected word with other random words 10% of the time and leaving the word unchanged 10% of the time. As a result, the transformer encoder has no prior knowledge about which words to predict. (Devlin et al. [2018]).

Task2: Next Sentence Prediction (NSP) The relationship between two sentences is crucial for the language models to capture for some downstream tasks such as question-answering and natural language inference. BERT is pre-trained using two types of examples in the NSP task where in the 50% of the examples the actual sentence B follows the previous sentence A and in another 50% of the cases the sentence A is followed by some random corpus. This task helps the model learn to understand when two sentences are connected and when they are not(Devlin et al. [2018]).

Fine-tuning of BERT The self-attention mechanism of the transformers helps BERT to fine-tune easily for many downstream tasks. During the fine-tuning, the pre-trained parameters are first initialized. Fine-tuning involves labeled data. Each downstream task has its own fine-tuned bert model. Specific layers are added on top of the BERT model tailored to each task. For example, for text classification, a dense layer is added on top of the BERT model with the appropriate number of output classes. Then the model is trained using an appropriate optimizer and loss function. The weights are updated by backpropagation algorithm. The fine-tuned model is evaluated on the validation and test sets. Next, the hyperparameters such as the learning rate, batch size and the number of epochs are tuned to optimize performance on the validation set. However fine-tuning a BERT model can be a computationally intensive process and may require access to powerful GPUs or TPUs (Devlin et al. [2018]).

BERT model choice The dbmdz/bert-base-german-uncased is a pre-trained BERT model for German language processing developed by the German research center for language technology called Deutsches Forschungszentrum für Künstliche Intelligenz (DFKI) in collaboration with the German Federal Ministry of Education and Research (BMBF). This model is based on the original BERT architecture and has been trained on a large corpus of German language text. The "uncased" in the model name indicates

that the model uses lower-cased text and does not differentiate between capitalized and non-capitalized words (hugging face [2023]). The bert-base-german-cased is another pre-trained BERT model for German language processing developed by the same research center(hugging face [2023]). The key difference between bert-base-german-cased and bert-base-german-uncased is that the former retains the capitalization of words in the text, whereas the latter converts all text to lowercase. This means that Bert-base-german-case is able to distinguish between capitalized and non-capitalized words. Both of these models are available for free and are used for various German language processing tasks.

3.3 Natural Language Toolkit - NLTK

NLTK (Natural Language Toolkit) is a popular open-source library for natural language processing in Python. It provides a wide range of tools and resources for tasks such as tokenization, stemming, part-of-speech tagging, and syntactic parsing. NLTK is widely used in academic and research settings, as well as in industry, and it is considered one of the foundational libraries for NLP in Python. Additionally, NLTK includes a large collection of corpora which are large bodies of text that can be used for training and testing NLP models (nltk [2023]).

Lemmatization Lemmatization is a popular text-preprocessing method in NLP that includes stripping words down to their most fundamental or root form or lemma. For example, the lemma of "walking" is "walk" and the lemma of "mice" is "mouse". Lemmatization differs from another text-preprocerssing method stemming as it tries to reduce word length but still keeps the exact meaning. For example, stemming reduces the word "better" to "bett" or "bet" while in lemmatization the lemma reduces to be "good" which makes sense. Stemming is often used in NLP application such as sentiment analysis whereas lemmatization is more useful in task s like text classification and information retrieval (Srinidhi [2023]).

3.4 Transformer-interpret

Explainable AI is a vital subfield in machine learning as it allows for the interpretation of the inner workings of complex models. Despite the trend toward prioritizing model complexity over interpretability in state-of-the-art research, there are ongoing efforts to

develop explainable AI solutions. Transformers Interpret is one such specialized tool for model explainability that is tailored to work with the huggingface transformers package. Its objective is to simplify the interpretation of any transformers model by enabling the explanation of any model in just two lines of code. Additionally, visualizations are provided in the form of notebooks as well as savable png and html files. SequenceClassificationExplainer is an explainer that is compatible with all sequence classification models from the Transformers package. It computes attributions for each label in the model's configuration and returns a dictionary of word attributions for each label. The visualize() method of it displays a table of attributions calculated per label. The resulting dictionary of word attributions maps labels to a list of tuples for each word and its corresponding attribution score. However, in cases where there is a large amount of text, numerical attributions can be challenging to read. To address this, the visualize() method utilizes Captum library's in-built visualization library to create an HTML file that highlights the attributions (cdpierse [2022]).

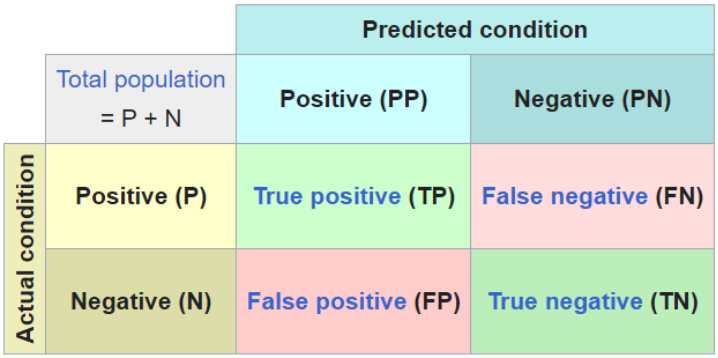
3.5 FastText

FastText is an open-source, free-to-use library developed by Facebook AI Research for text classification and word embedding. The key concept behind FastText is the use of n-grams which are contiguous sequences of n words in a text to capture the meaning and context of words in a sentence. For example, in the sentence "The quick brown fox jumps over the lazy dog" some examples of n-grams are "The quick brown" "over the lazy" "brown fox jumps" etc. FastText first breaks down words into their constituent character n-grams and then learns continuous representations for each n-gram using a neural network. These representations are then used to learn a model for text classification or to perform other NLP tasks like named entity recognition and machine translation. FastText can learn text representations for both individual words and n-grams which helps in capturing the meaning of rare or unknown words. Additionally, FastText can handle multiple word meanings by breaking down words into multiple character n-grams and also uses subword information to handle out-of-vocabulary words (AI [2015]).

3.6 Confusion matrix

The confusion matrix is a visualization tool used to evaluate the performance of a classification model in machine learning. It provides a tabular representation of the model's

predictions and actual outcomes. Each row of the matrix represents the instances from a true label while each column represents the instances from a predicted label or vice versa. The diagonal elements represent the cases where the predicted label is equal to the true label and the off-diagonal elements are those mislabeled by the classifier. The accuracy of the model can be inferred by the higher values along the diagonal of the matrix indicating a higher number of correct predictions made by the classifier. The confusion matrix thus helps us to determine what types of errors the classifier is making and how well it is performing on the classification task.



Source: Wikiepedia [2023]
Figure 6: Confusion matrix with rows contain the actual label and columns the predicted label.

In a two-by-two confusion matrix, there are four possible outcomes given a classifier and an instance. If both the instance and its predicted value are positive, it is considered as true positive (TP), if the prediction is negative, it is false negative (FN). When the instance is negative and is classified as negative, then it is true negative (TN), if it is classified as positive, then it is false positive (FP). Several common metrics are calculated from the confusion matrix such as

$$\text{Recall} = \frac{TP}{TP + FN}; \quad \text{TN rate} = \frac{TN}{TN + FP}$$

$$\text{Precision} = \frac{TP}{TP + FP}; \quad \text{Accuracy} = \frac{TP + TN}{P + N}; \quad \text{F-1 score} = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}}$$

The F-1 score is a metric that combines precision and recall by taking their weighted average. An increase in precision typically results in a decrease in recall and the F-1

score offers a balanced assessment of both metrics. However, the F-1 score should not be used in isolation, as it does not provide information about which metric is being maximized.

The term "confusion matrix" originates from its function of displaying whether a classification model is making correct predictions or not. The confusion matrix can be extended to multi-class classification problems and is especially helpful in cases where the number of classes is imbalanced. In such cases, relying solely on classification accuracy can be misleading if one class dominates the dataset.

4 Result analysis

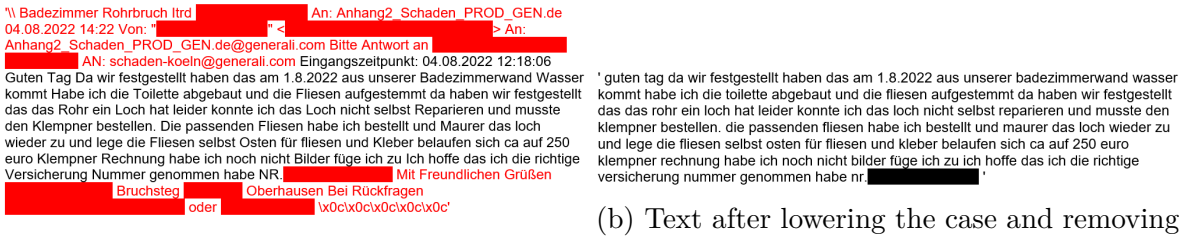
The BERT model is trained first with fully processed text and then with lightly processed text. The preprocessing steps discuss how these two types of preprocessed text are generated. After that, the distribution of words in the text is analyzed. Next, the BERT token explainer is used to show how word attribution scores contribute to the model prediction. Finally, the predictions made by both the BERT model and the FastText model for the same field are presented followed by a comparison of their performances.

4.1 Preprocessing of the data

Fully processed text The first step is to preprocess the text for cleaning the raw text. For this purpose the Natural Language Toolkit (NLTK) is used which includes deletion of some parts of the text which doesn't add value to the text. The cleaning steps in this project includes lowering case (reduces the dimension of the vocabulary), removing header and footer from email. Figure 7(a) shows the raw text with header and footer highlighted in red and figure 7(b) displays the text after the header and footer being removed.

Next, any special characters, single characters and digits are removed from the text. Additionally, any multiple spaces are replaced with a single space as shown in figure 8 and after that the resulting text is lemmatized. In figure 8(a), the words highlighted in red are not being considered after lemmatization as shown in figure 8(b).

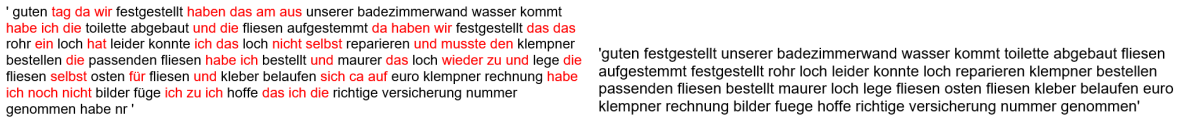
German contraction words if present any are also expanded. For e.g., "am" is replaced with "an dem". Similarly, German special characters are substituted like 'ä' and with



(a) Raw text

(b) Text after lowering the case and removing the header and footer of the mail.

Figure 7: Preprocessing of text.

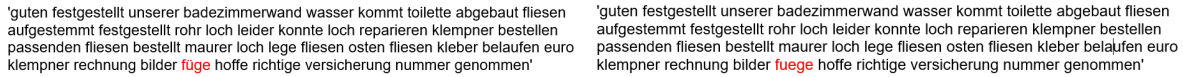


(a) Befor lemmatization.

(b) After lemmatization.

Figure 8: Preprocessed of text.

'ae' and 'ü' with 'ue'. As shown in figure 9(a), the word 'füge' (in red) is expanded as 'fuege' in figure 9(b). Finally, the text undergoes another round of lemmatization. The length 1033 of the raw text is thus reduced to 326 after the preprocessing.



(a) Befor lemmatization.

(b) After lemmatization.

Figure 9: Preprocessed of text.

Lightly preprocessed text The text is not completely preprocessed in this step. The goal is to check how BERT performs on almost raw text. The text is left in its raw form but the header and footer of the email are removed so are the special characters and single characters as well. Multiple spaces are replaced by a single space. The key difference between the fully processed text and the lightly processed text is that the latter does not undergo rounds of lemmatization. Figure 10(b) shows the lightly processed text. The length of which is being reduced to 435.

Figure 11 explores the distributions of the number of words in the raw text whereas figure 12 shows the same after both the preprocessing. The distribution in the preprocessed text indicates that the number of word counts has decreased after both the preprocessing steps.

'\\ Badezimmer Rohrbruch Itrd [redacted] An: Anhang2_Schaden_PROD_GEN.de 04.08.2022 14:22 Von: "[redacted]" <[redacted]> An: Anhang2_Schaden_PROD_GEN.de@generali.com Bitte Antwort an [redacted] AN: schaden-koeln@generali.com Eingangszeitpunkt: 04.08.2022 12:18:06 Guten Tag Da wir festgestellt haben das am 1.8.2022 aus unserer Badezimmerwand Wasser kommt Habe ich die Toilette abgebaut und die Fliesen aufgestemmt da haben wir festgestellt das das Rohr ein Loch hat leider konnte ich das Loch nicht selbst Reparieren und musste den Klempner bestellen. Die passenden Fliesen habe ich bestellt und Maurer das loch wieder zu und lege die Fliesen selbst Osten für fliesen und Kleber belaufen sich ca auf 250 euro Klempner Rechnung habe ich noch nicht Bilder füge ich zu Ich hoffe das ich die richtige Versicherung Nummer genommen habe NR.[redacted] Mit Freundlichen Grüßen [redacted] Bruchsteg [redacted] Oberhausen Bei Rückfragen [redacted] oder [redacted] \x0c\x0c\x0c\x0c\x0c'

(a) Raw text.

'Guten festgestellt haben 1.8.2022 unserer Badezimmerwand Wasser kommt Habe Toilette abgebaut Fliesen aufgestemmt haben festgestellt Rohr Loch leider konnte Loch nicht selbst Reparieren musste Klempner bestellen. passenden Fliesen habe bestellt Maurer loch wieder lege Fliesen selbst Osten fliesen Kleber belaufen sich euro Klempner Rechnung habe noch nicht Bilder füge hoffe richtige Versicherung Nummer genommen habe NR.[redacted]'

(b) Lightly peprocessed text

Figure 10: Lightly preprocessed text

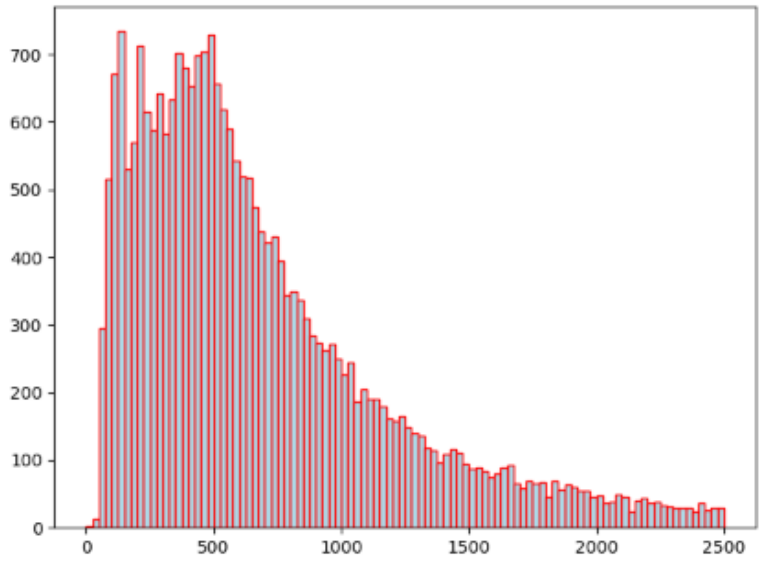
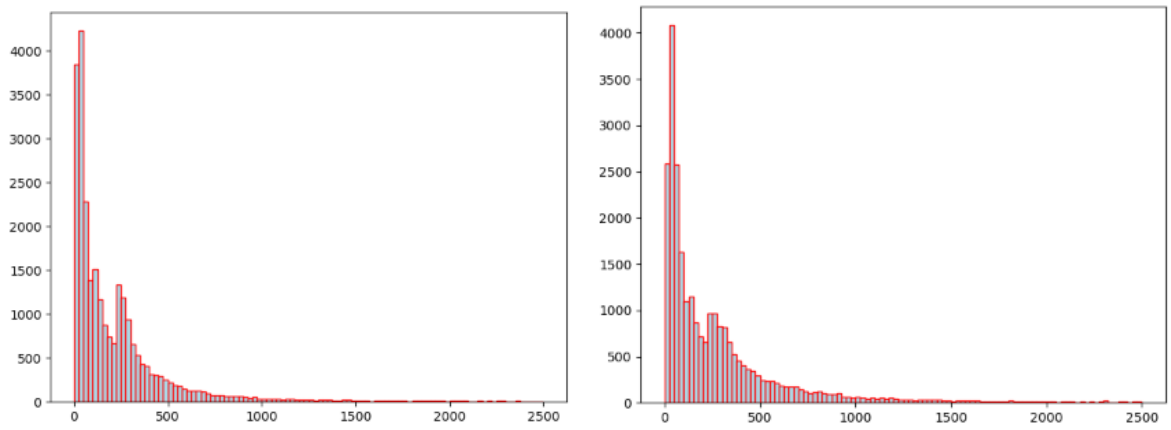


Figure 11: Word counts distribution in the raw text.

4.2 Analysis of contributions of word attributions in lightly processed text and fully preprocessed text

To investigate the impact of different preprocessing methods on the BERT model prediction, a comparison is made between the lightly and fully preprocessed texts for classifying documents under LW and ST categories. The SequenceClassificationExplainer class from the transformer-interpret library is used to analyze the documents. This class takes as input the fine-tuned BERT model, corresponding tokenizer, and the custom labels. Attention scores of each token in the text are retrieved and visualized. If the length of the preprocessed text exceeds the limit of 512 tokens after encoding, the sentence is truncated. Positive attribution scores indicate a positive contribution of the token towards the prediction, while negative attribution scores indicate a negative impact.



(a) Fully processed text.

(b) Lightly processed text.

Figure 12: Word counts distribution in the processed text.

Tokens with higher attribution scores appear greener, while those with lower attribution scores appear redder. The results of the two preprocessing methods are then compared.

Example documents of class LW The important words are shown as highlighted for the lightly preprocessed text are presented in Figure 13 and table 1 lists their corresponding attribution scores. The actual label is LW which is encoded as zero. However, the model predicts the label as ST, which is incorrect. The word importance column in the figure 13 indicates why this misclassification occurred. The highest contributor towards the attribution score is "Dach" (in deep red) which is a negative token for the prediction. In contrast, words like "Wassers" (in light green), "ch", and "aden" provide less positive attribution scores towards the prediction. The scores for "Wassers", "ch", and "aden" are 0.08, −0.01 and 0.02 respectively, whereas "Dach" provides −0.94 as a score resulting in the misclassification.

Table 1: Attention scores of the tokens in the lightly processed text.

'[CLS]'	'Sehr'	'geehrt'	'Wassers'	'ch'	'aden'	'rohr'	'Dach'	'ent'	'[SEP]'
0,0	-0,01	0,01	0,08	-0,01	0,02	0,16	-0,94	0,00	0,0

After the text is completely processed, it can be seen that the words such as "wasser" and "schaden" have been properly tokenized. As a result, the tokens of these words provide positive attribution scores for the correct prediction, which is LW. Figure 14 displays the highlighted words and table 2 lists the corresponding attribution scores for these tokens.

Legend: ■ Negative □ Neutral ■ Positive

True Label	Predicted Label	Attribution Label	Attribution Score	Word Importance
0	ST (0.07)	LW	-0.25	[CLS] Sehr geehrt ##e Damen Herren , Versicherungs ##nehmer ##in meldet einen Wassers ##ch ##aden 14 . 07 . 20 ##21 . Beding ##t durch Stark ##regen Schaden ##tag vermuten derzeit einen Rohr ##bruch Ab ##leitungs ##rohr Dach ##ent ##wässer ##ung Gar ##age . Ent ##wässer ##ung Gar ##age wird unter Gar ##age Straßen ##anschluss geführt . Gar ##age grenzt Teil Wohnhaus ##es , sich auch gesch ##äd ##igte Gäste ##zimmer befindet . Wasser drückt sich oder unter Boden ##platte Gäste ##zimmer . bitten Vergabe einer Schaden ##nummer Freigabe einer Le ##ck ##age ##ort ##ung . Viele ##n Dank . Gern ##e stehen Ihnen Rück ##fragen Verfügung . [SEP]

Figure 13: The impacts of the attribution scores of tokens in the lightly processed text.

As seen in table 2, the token "wasser" has 0.30 and "schaden" has 0.37 as attribution scores respectively. The positive scores help make the model correct prediction in this text.

Legend: ■ Negative □ Neutral ■ Positive

True Label	Predicted Label	Attribution Label	Attribution Score	Word Importance
0	LW (0.80)	LW	0.53	[CLS] geehrt ##e dam ##en herr versicher ##ungs ##nehmer ##in meldet wasser ##schaden bedingt stark ##regen schaden ##tag vermuten derzeit ro ##hr ##bruch able ##itu ##n ##gs ##rohr da ##chen ##t ##wa ##esse ##rung gar ##age ent ##wa ##esse ##rung gar ##age gar ##age stra ##ssen ##anschluss gef ##ue ##hr ##t gar ##age grenzt teil wohn ##hauses gesch ##ae ##digte g ##ae ##ste ##zimmer befindet wasser dr ##ue ##ckt bo ##den ##platte g ##ae ##ste ##zimmer bitten verg ##abe schaden ##nummer frei ##gabe le ##ck ##age ##ort ##ung vielen dank gerne stehen ru ##eck ##fragen verf ##ue ##gung [SEP]

Figure 14: The impacts of the attribution scores of tokens in the fully processed text.

Table 2: Attention scores of the tokens in the fully processed text.

'[CLS]'	'geehrt'	'Wassers'	'schaden'	'stark'	'regen'	'derzeit'	'ro'	'hr'	'[SEP]'
0,0	0,02	0,30	0,37	-0,20	-0,74	0,10	0,26	0,22	0,0

Example documents of class ST In figure 15, the true label is ST (encoded as 1) but the model predicts a different class. The "Word Importance" column highlights "Strum"

in bright green and "Tage" in light red, while "Glas" is shown in deep red. The scores in table 3 indicate that "Glas" has more negative scores ($-0,39$ and $-0,32$) compared to the positive score $0,082$ of "Strum". Hence the predicted class is other claims not ST.

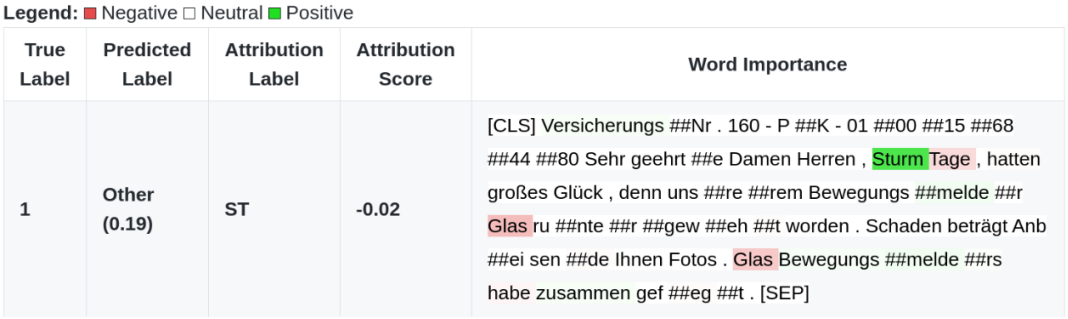


Figure 15: The impacts of the attribution scores of tokens in the lightly processed text.

Table 3: Attention scores of the tokens in the lightly processed text.

'[CLS]'	'Sturm'	'Tage'	'Glas'	'Glas'	'habe'	'[SEP]'
0,0	0,082	-0,21	-0,39	-0,32	-0,07	0,0

In case of the fully preprocessed text, the words are tokenized differently. The word "Glas" is split into two tokens: "gl" and "as" and "Strum" into "st" and "urm". Figure 16 shows the highlighted words which are "st", "urm" (in deep green), "as" and "gl" (in red). Table 4 provides the attribution scores for these words. The score for "urm" is the highest positive score at 0.73 and "st" has a score of 0.21. These two scores together have more weightage towards prediction than those of "gl" and "as" which is $0,35$ as combined. As a result, the model correctly predicts the ST class using the fully preprocessed text. More such examples for fully processed text can be found in Appendix.

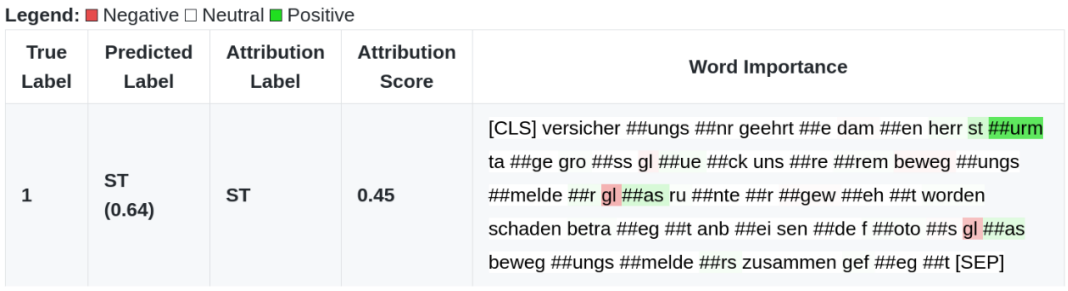


Figure 16: The impacts of the attribution scores of tokens in the fully processed text.

Table 4: Attention scores of the tokens in the fully processed text.

'[CLS]'	'st'	'urm'	'gl'	'gl'	'as'	'gl'	'as'	'[SEP]'
0,0	0,21	0,73	0,09	-0,44	0,18	0,36	0,16	0,0

4.3 Model performances

4.3.1 Parameters for fine-tuning BERT

The dataset, consisting of 26,468 records, was split into two parts for testing and validation purposes. The test dataset was allocated 20% of the data while the validation dataset received 25% of the data. As a result, the test and validation datasets consist of 15,880 and 5,594 records respectively. The three claims are displayed along with their total corresponding support in figure 17 whereas figure 18 gives a brief view of the original records.

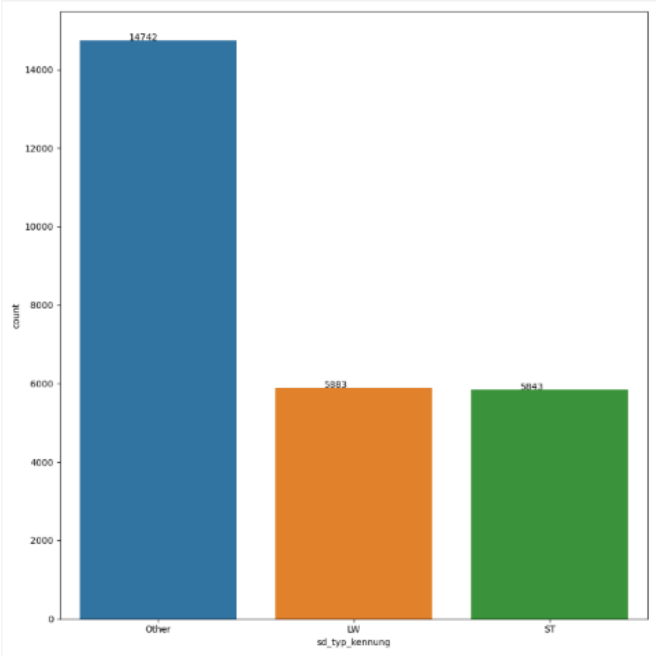


Figure 17: Categorical distribution of the predicted field.

To train the BERT model, two different types of tokenizers and models from the Hugging Face library were utilized. The BERT-base-german-uncased model was used for the fully processed text which contains only lower case letters. For the lightly processed text, BERT-base-german-cased was used instead. Parameters for training BERT model are

	doc_id	sd_anlage_dat	raw_text	schaden_objekt	sd_typ_kennung	sd_urs_art	schadentag	notifier	processed_text
0	079994I...	2021-11-11	\'r Neusch...	WG	LW	2	27-05-2021	AD	geehrte damen h...
1	000990...	2022-12-20	20 Dez 22,...	Other	LW	2	19-12-2022	VN	immobiliendiens...
2	079994I...	2021-08-18	Die Repro...	WG	LW	2	12-08-2021	AD	geehrte damen h...
3	079994I...	2022-06-22	Neuschad...	WG	LW	2	02-06-2022	AD	geehrte damen h...
4	079994I...	2022-10-24	vedr [EXT...	WG	LW	0	22-10-2022	AD	externe mail bitt...
5	079994I...	2021-09-27	2-12.697....	WG	LW	0	25-09-2021	VN	geehrter herr kle...

Figure 18: The snapshot of the dataframe.

set following the guidelines of the original paper (Devlin et al. [2018]). Table 5 outlines the parameters that were used for training the BERT model.

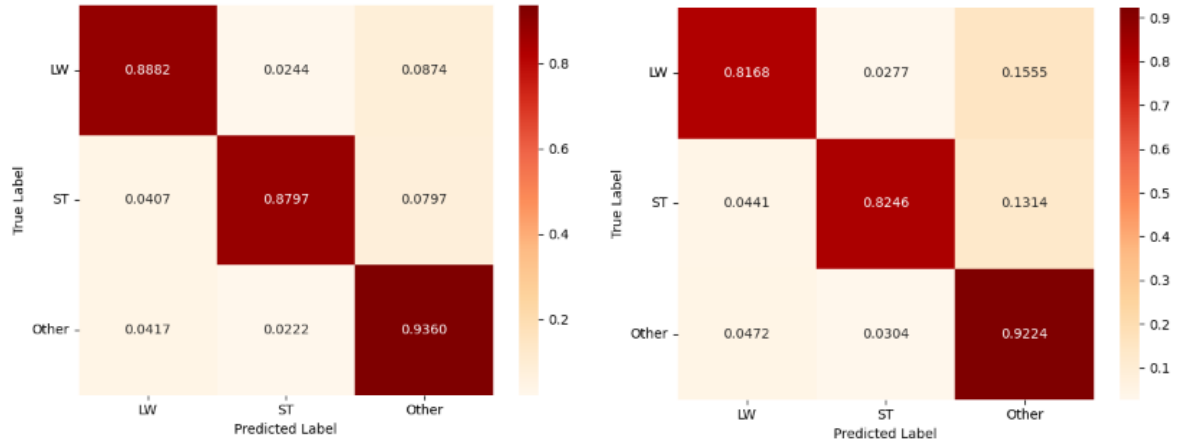
Table 5: Parameter lists for fine-tuning BERT.

Max length of token	512
Batch size	32
Epochs	3
Learning rate	2e-05
Optimizer	AdamW
Optimizer learning rate	5e-5

4.3.2 Using lightly processed text

The normalized confusion matrix in Figure 19 displays the performance of BERT and FastText models while being fine-tuned on lightly preprocessed text. Each row of the matrix represents the true labels and each column indicates the predicted labels. The diagonal elements show the correctly predicted labels by the models. The highest percentage of correct predictions for the other class is achieved by both models, which is around 93.60% for BERT and 92.24% for FastText. This class has the most support. However, BERT model considerably outperforms FastText in predicting the ST and LW classes. BERT predicts 87.97% for ST label and 88.82% for LW label, whereas FastText can only predict 82.46% and 81.68% respectively. The lighter shade of red for LW and ST class shows that their supports are lower than that of the other class. All non-diagonal elements represent the mismatched actual and predicted label combinations but the percentage of wrongly classified samples is relatively low. Therefore, for training with the lightly preprocessed text, BERT outperforms the FastText model.

The merged classification report of BERT and FastText models is presented in Table 6. The results show that BERT model outperforms the FastText model in terms of precision, f-1 scores and accuracy. For example, BERT achieves a precision of 86% for



(a) BERT model. (b) Fasttext model

Figure 19: Confusion matrix of models using lightly processed text.

LW class, while FastText only achieves 84%. Similarly, BERT achieves a precision of 92% and 93% for ST and other classes respectively, while FastText achieves 89% for both the classes. The f-1 scores of both models are close to 1, indicating good performance but BERT achieves higher f-1 scores than FastText for all three classes. The support column indicates the number of occurrences of each class in the test dataset. BERT model accuracy is 91%, while for FastText it is 88%. Therefore, BERT model is considered to be a better performer than the FastText model.

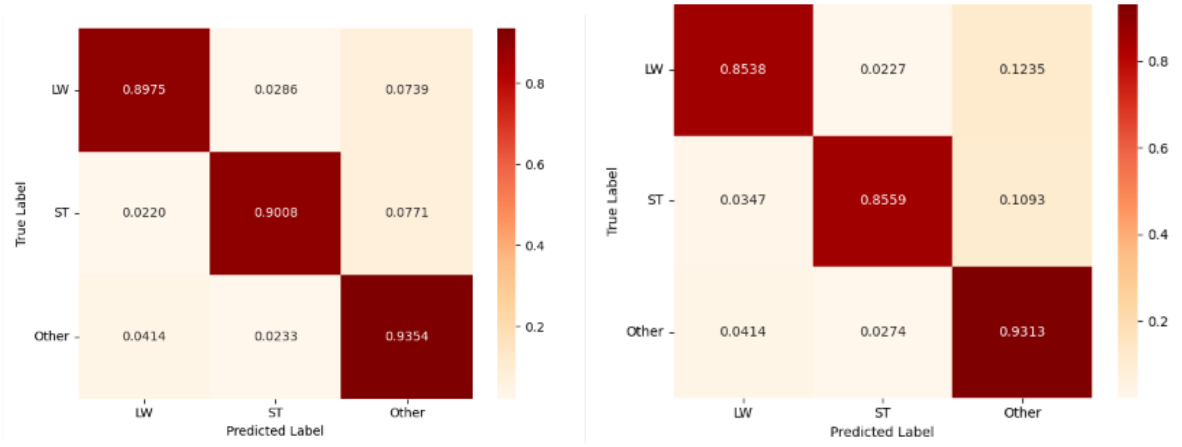
Table 6: Classification report of models using lightly processed text.

	BERT			FastText			
	precision	recall	f-1 score	precision	recall	f-1 score	support
LW	0,86	0,89	0,87	0,84	0,82	0,83	1190
ST	0,92	0,88	0,90	0,89	0,82	0,86	1180
Other	0,93	0,94	0,93	0,89	0,92	0,90	2924
Accuracy	0,91			0,88			

4.3.3 Using fully preprocessed text

The normalized confusion matrix in Figure 20 shows the performance of models using fully preprocessed text. As in previous cases, the other class (right-most diagonal element) has the highest percentage of correct classification i.e. $93,54\%$ for BERT and $93,13\%$ for FastText. ST class has the second highest percentage of correct classification

with 90.08% for BERT and 85.59% for fasttext. The LW class has the lowest percentage of correct classification with 89.75% for BERT and 85.38% for fasttext. Overall, BERT achieves higher scores than fasttext in correctly classifying LW and ST classes in this case also.



(a) BERT model.

(b) FastText model

Figure 20: Confusion matrix of models using fully processed text.

Table 7 presents the classification report for both models on the fully preprocessed text. BERT's precision scores for the LW, ST and other classes are 88%, 91% and 94% respectively. On the other hand, FastText achieves precision scores of 86%, 90% and 91% for the respective classes. Both models have f-1 scores close to 1 indicating good performance. However, BERT again outperforms FastText in the classification. The model accuracies are 91% and 90% for BERT and FastText respectively.

Table 7: Classification report of models using fully processed text.

	BERT			FastText			
	precision	recall	f-1 score	precision	recall	f-1 score	support
LW	0,88	0,90	0,89	0,86	0,85	0,86	1190
ST	0,91	0,90	0,91	0,90	0,86	0,88	1180
Other	0,94	0,93	0,94	0,91	0,93	0,92	2924
Accuracy	0,91			0,90			

5 Summary

As part of the masters program in data science, an external internship project can replace module project work - 4. I've had the opportunity to work as a student assistant with the Data Analytics team at Generali Deutschland AG. The team has implemented an automatic pipeline to automate the claims notification process. The project aims to classify five fields: schadentag, schaden-objekt, notifier, schaden-typ-kennung and schaden-urs-art properly using different text classifiers, such as FastText, SpaCy and an ensemble model. The classifiers are trained and evaluated using preprocessed text and labels from the HIVE database. The data engineers in the team are currently working on deploying the production pipeline.

This project report focuses on the prediction of only the schaden-typ-kennung field using the pre-trained BERT model. The schaden-typ-kennung field has three subcategories: leitungswasser (LW), storm (ST), and other claims (including fire, glass, bike theft, etc.). There are uneven support in each claims subcategories. 25% of the data is split for validation and 20% for testing. Two different types of text preprocessing are performed on the raw text before training: full preprocessing which involves different text-preprocessing steps whereas light processing removes only the header and footer from the raw text. Two BERT models and their corresponding tokenizers are used for training on the two preprocessed texts. The reason for using two preprocessing methods is to separately evaluate the performances of the BERT model for further consideration.

The pre-trained BERT models can only handle the maximum input sequence length of 512. In this project, text longer than 512 are hence truncated. The Transformer-interpret package is used to generate word attribution scores and visualize the tokens that contributes to the model's predictions. The analysis shows that the fully preprocessed text contributes more to correct predictions than the lightly preprocessed text and the model achieves an overall accuracy of 91% in both the cases. However, the model precision is higher for the fully preprocessed text. The performance of the BERT model is compared to that of the FastText model. The FastText model achieves an accuracy of 88% and 90% for the lightly and fully preprocessed texts respectively. Clearly BERT model outperformes the FastText model.

The training of each BERT model has required almost 30 hours to complete using the computational power of CPUs. The BERT model was not fine-tuned in this project, as it can be a computationally intensive process that requires access to powerful GPUs

or TPUs. Future plans involve expanding the sub-categories of schaden-typ-kennung into individual models to ensure even classes for each prediction and to train the BERT model on other business fields. Also to find a way out to deal with the text longer than 512 for accurate predictions. Additionally, this year Generali plans to move all the trained models to third-party cloud services Amazon Web Services (AWS). The plan also involves to train the BERT model on AWS cloud for faster and efficient training and add the model performance in the ensemble model for the claims classification process.

Bibliography

Meta AI. Fasttext, 2015. URL https://ai.facebook.com/tools/fasttext/. Visited on 24th March 2023.

Jay Alammar. Visualizing a neural machine translation model (mechanics of seq2seq models with attention), 2018. URL https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2 Visited on 14th March 2023.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Jason Brownlee. What are word embeddings for text?, 2017. URL https://machinelearningmastery.com/what-are-word-embeddings/. Visited on 14th March 2023.

cdpierse. Transformers interpret, 2022. URL https://github.com/cdpierse/transformers-interpret/blob/master/README.md. Visited on 24th March 2023.

Google code archive. word2vec, 2013. URL https://code.google.com/archive/p/word2vec/. Visited on 14th March 2023.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018.

hugging face. dbmdz/bert-base-german-uncased, 2023. URL https://huggingface.co/bert-base-german-cased. Visited on 24th March 2023.

nltk. Natural language toolkit, 2023. URL https://www.nltk.org/. Visited on 24th March 2023.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation, 2014. URL https://nlp.stanford.edu/projects/glove/. Visited on 14th March 2023.

Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*, 2018.

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

Sunny Srinidhi. Lemmatization in natural language processing (nlp) and machine learning, 2023. URL `https://builtin.com/machine-learning/lemmatization`. Visited on 24th March 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wikiepedia. Confusion matrix, 2023. URL `https://en.wikipedia.org/wiki/Confusion`$_m atrix/.Visi$

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27, 2015.

Appendix

Table 8: Attention scores of the tokens in the fully processed text.

’[CLS]’	’entstanden’	’raum’	’immel’	’kinder’	’autos’	’itz’	’[SEP]’
0,0	0,21	0,22	0,68	-0,31	0,52	0,15	0,0

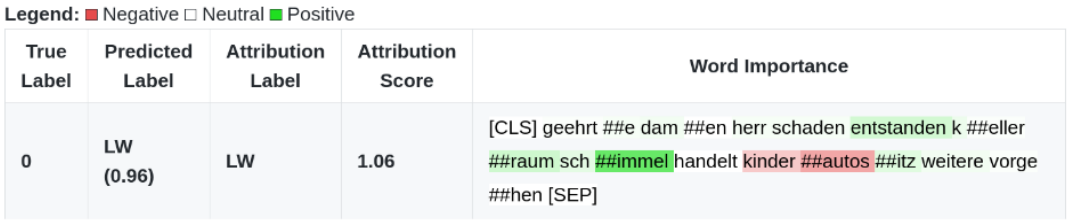


Figure 21: The impacts of the attribution scores of tokens in the fully processed text.

Table 9: Attention scores of the tokens in the fully processed text.

’[CLS]’	’linke’	’play’	’play’	’icken’	’eien’	’wasser’	’schaden’	’[SEP]’
0,0	-0,05	0,05	-0,02	-0,05	-0,14	0,94	0,28	0,0

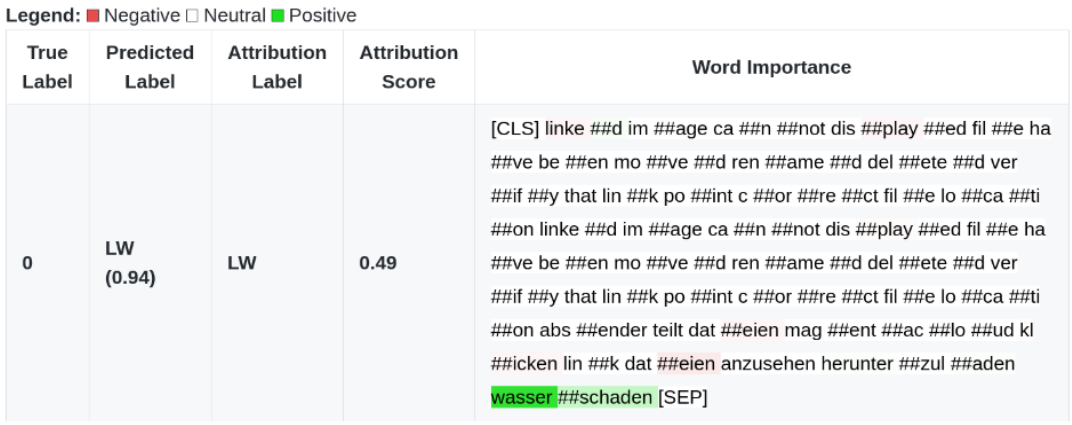


Figure 22: The impacts of the attribution scores of tokens in the fully processed text.

Table 10: Attention scores of the tokens in the fully processed text.

’[CLS]’	’melden’	’st’	’rom’	’rasse’	’ach’	’besch’	’digt’	’verhalten’	[SEP]’
0,0	0,34	0,37	0,52	0,21	0,23	0,17	0,29	-0,40	0,0

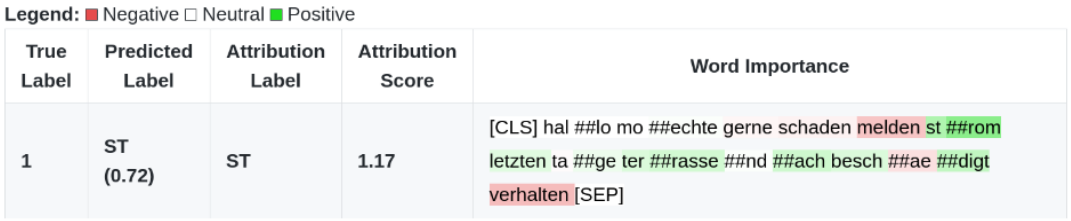


Figure 23: The impacts of the attribution scores of tokens in the fully processed text.

Table 11: Attention scores of the tokens in the fully processed text.

'[CLS]'	'ha'	'gel'	'wetter'	'schw'	'imm,'	'besch'	'prech'	[SEP]'
0,0	0,51	0,58	0,36	0,32	-0,21	0,09	0,05	0,0

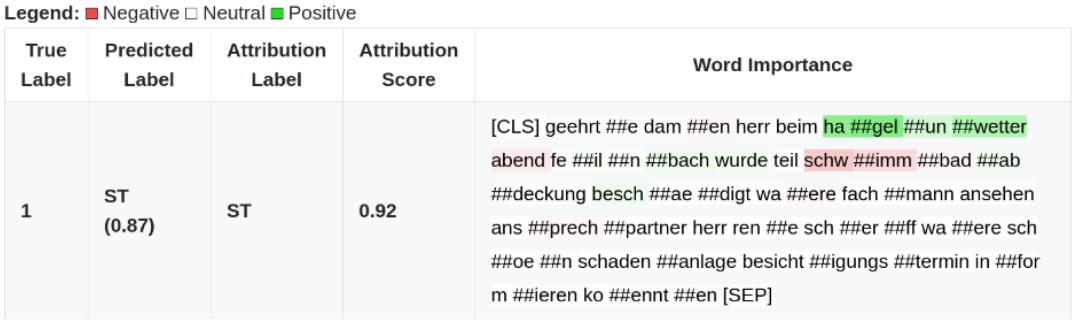


Figure 24: The impacts of the attribution scores of tokens in the fully processed text.